

SQL Scripts

Case: ICRC – “Where we Work”

Source Site: [Explore our global reach | International Committee of the Red Cross](#)

1.Introduction

The part aims to demonstrate the concrete implementation of the database system supporting the International Committee of the Red Cross (ICRC) "Where We Work" web product. The core content includes a complete set of SQL scripts, covering multiple aspects from database schema creation to complex business queries.

By executing these scripts, a complete database environment can be constructed and validated for its effectiveness in supporting the website's core functionalities, such as browsing operations by region and country, managing relevant content, and analyzing operational data. This deliverable serves to materialize and put into practice the earlier ERD design.

2.SQL Code:

```
1.      -- search country relate infomation
2.      -- keypoints : LEFT, RIGHT join
3.      SELECT
4.      t1.continent_name
5.      ,t2.country_name
6.      ,t3.icrc_name
7.      ,t4.user_name
8.      ,t5.article_title
9.      ,t6.`desc`
10.     FROM
11.     CONTINENT t1 RIGHT JOIN COUNTRY t2 on t1.continent_id = t2.continent_id
12.     LEFT JOIN ICRC t3 on t2.country_id = t3.country_id
13.     LEFT JOIN `USER` t4 on t2.country_id = t4.country_id
14.     LEFT JOIN ARTICLE t5 on t5.author_id = t4.user_id
15.     LEFT JOIN COUNTRY_INFO t6 on t2.country_id = t6.country_id;
16.     ;
17.
18.     -- who write over one ARTICLES
19.     -- Key points: subquery , join
20.     SELECT
21.     t3.user_name
22.     ,t3.user_country
23.     ,t4.country_name
24.     FROM
25.     (
26.     SELECT t1.user_name,t1.user_country
27.     FROM `USER` t1
```

```

28. WHERE t1.user_id in (SELECT author_id from `ARTICLE` GROUP BY author_id HAVI
NG count(1)>1)
29. )
30. t3 LEFT JOIN COUNTRY t4 on t3.user_country = t4.country_id
31. ;
32.
33. -- The person who writes the most articles with name
34. -- rank join agg
35. select
36. t2.user_name,count(1) as 'article num '
37. FROM ARTICLE t1 LEFT JOIN `USER` t2 on t1.author_id = t2.user_id
38. GROUP BY t1.author_id
39. order by count(1) desc
40. limit 100
41. ;
42.
43. -- top 10 images user in articles
44. -- agg join subquery
45. SELECT
46. t2.*
47. ,t3.user_name
48. FROM
49. (
50. SELECT
51. author_id
52. ,sum(JSON_LENGTH(page_img_list)) as image_total_per_person
53. FROM ARTICLE t1
54. GROUP BY author_id
55. ORDER BY sum(JSON_LENGTH(page_img_list)) desc
56. )t2 LEFT JOIN `USER` t3 ON t2.author_id = t3.user_id
57. LIMIT 10;
58.
59.
60.
61. -- Per CONTINENT populaiton avg contribution top3
62. -- string operate, json operate, join, subquery, agg ,VIEW
63.
64. -- Contributions in United States dollars per country
65. CREATE VIEW COUNTRY_CONTRIBUTE_USD_VIEW as
66. SELECT
67. t1.country_id,
68. t1.country_name,
69. t1.continent_id,
70. t2.statistics
71. ,JSON_EXTRACT( t2.statistics, '$.population') as 'population'
72. ,JSON_EXTRACT( t2.statistics, '$.annual_aid_amount') as 'annual_aid_amount'
73. ,(SUBSTRING_INDEX(JSON_UNQUOTE(t2.statistics-
>'$.annual_aid_amount'),' ', 1) + 0)
74. * CASE
75. WHEN JSON_UNQUOTE(t2.statistics-
>'$.annual_aid_amount') LIKE '%M%' THEN 1000000
76. WHEN JSON_UNQUOTE(t2.statistics-
>'$.annual_aid_amount') LIKE '%B%' THEN 1000000000
77. END
78. / e.usd_exchange_rate as aid_usd

```

```

79. FROM
80. COUNTRY t1 LEFT JOIN COUNTRY_INFO t2 ON t1.country_id = t2.country_id
81. LEFT JOIN EXCHANGE_RATES e
82. ON SUBSTRING(SUBSTRING_INDEX(JSON_EXTRACT(t2.statistics,'$.annual_aid_amo
ount'), ',' -
1), 1, LENGTH(SUBSTRING_INDEX(JSON_EXTRACT(t2.statistics,'$.annual_aid_amount'), ',' -
1)) - 1) = e.currency_code
83. ;
84.
85. SELECT
86. t3.continent_id,
87. t4.continent_name
88. ,sum(aid_usd) as aid_total
89. ,sum(population) as population_total
90. ,sum(aid_usd)/sum(population) as avg_aid_per_person
91. FROM
92. COUNTRY_CONTRIBUTE_USD_VIEW t3 left join CONTINENT t4 on t3.continent_id =
t4.continent_id
93. GROUP BY continent_id
94. order by avg_aid_per_person desc
95. limit 3
96. ;

```

3.SQL Query Analysis

3.1 Query 1: Search country-related information

local_mysqldbms805					
<pre> 1 -- search country relate information 2 3 -- keypoints: 1 LEFT, RIGHT join 4 5 SELECT 6 t1.continent_name, 7 t2.country_name, 8 t3.icrc_name, 9 t4.user_name, 10 t5.article_title, 11 t6.'desc' 12 FROM 13 CONTINENT t1 14 RIGHT JOIN COUNTRY t2 ON t1.continent_id = t2.continent_id 15 LEFT JOIN ICRC t3 ON t2.country_id = t3.country_id 16 LEFT JOIN 'USER' t4 ON t2.country_id = t4.country_id 17 LEFT JOIN ARTICLE t5 ON t3.author_id = t4.user_id 18 LEFT JOIN COUNTRY_INFO t6 ON t2.country_id = t6.country_id; 19 20 21 </pre>					
MessageSummaryResult 1Result 2Result 3Result 4Status					
continent_name	country_name	icrc_name	user_name	article_title	desc
Asia	China	ICRC Beijing Office	Zhang Wei	2023 China Red Cross Annual Report	China is the world's most populous country, with a rich history and diverse culture. The ICRC has multiple offices and relief centers a
Asia	China	ICRC Beijing Office	Li Na	(NULL)	China is the world's most populous country, with a rich history and diverse culture. The ICRC has multiple offices and relief centers a
Asia	China	ICRC Beijing Relief Center	Zhang Wei	2023 China Red Cross Annual Report	China is the world's most populous country, with a rich history and diverse culture. The ICRC has multiple offices and relief centers a
Asia	China	ICRC Beijing Relief Center	Li Na	(NULL)	China is the world's most populous country, with a rich history and diverse culture. The ICRC has multiple offices and relief centers a
Asia	China	ICRC Shanghai Office	Zhang Wei	2023 China Red Cross Annual Report	China is the world's most populous country, with a rich history and diverse culture. The ICRC has multiple offices and relief centers a
Asia	China	ICRC Shanghai Office	Li Na	(NULL)	China is the world's most populous country, with a rich history and diverse culture. The ICRC has multiple offices and relief centers a
Asia	Japan	ICRC Tokyo Office	Tanaka Yuki	2023 Japan Red Cross Disaster Response Summary	Japan is an island nation with advanced technology and a strong humanitarian sector. The ICRC operates in Tokyo, Osaka, and other
Asia	Japan	ICRC Tokyo Office	Tanaka Yuki	Japanese Community Health Promotion Project	Japan is an island nation with advanced technology and a strong humanitarian sector. The ICRC operates in Tokyo, Osaka, and other
Asia	Japan	ICRC Tokyo Medical Hub	Tanaka Yuki	2023 Japan Red Cross Disaster Response Summary	Japan is an island nation with advanced technology and a strong humanitarian sector. The ICRC operates in Tokyo, Osaka, and other
Asia	Japan	ICRC Tokyo Medical Hub	Tanaka Yuki	Japanese Community Health Promotion Project	Japan is an island nation with advanced technology and a strong humanitarian sector. The ICRC operates in Tokyo, Osaka, and other
Asia	Japan	ICRC Osaka Office	Tanaka Yuki	2023 Japan Red Cross Disaster Response Summary	Japan is an island nation with advanced technology and a strong humanitarian sector. The ICRC operates in Tokyo, Osaka, and other
Asia	Japan	ICRC Osaka Office	Tanaka Yuki	Japanese Community Health Promotion Project	Japan is an island nation with advanced technology and a strong humanitarian sector. The ICRC operates in Tokyo, Osaka, and other
Asia	South Korea	ICRC Seoul Office	Kim Min-ji	2023 South Korea Red Cross Annual Work Report	South Korea is a dynamic country in East Asia. The ICRC has offices in Seoul and Busan, focusing on disaster response, community v
Asia	South Korea	ICRC Seoul Office	Kim Min-ji	South Korea Typhoon Relief Operation (2023)	South Korea is a dynamic country in East Asia. The ICRC has offices in Seoul and Busan, focusing on disaster response, community v
Asia	South Korea	ICRC Seoul Relief Station	Kim Min-ji	2023 South Korea Red Cross Annual Work Report	South Korea is a dynamic country in East Asia. The ICRC has offices in Seoul and Busan, focusing on disaster response, community v
Asia	South Korea	ICRC Seoul Relief Station	Kim Min-ji	South Korea Typhoon Relief Operation (2023)	South Korea is a dynamic country in East Asia. The ICRC has offices in Seoul and Busan, focusing on disaster response, community v
Asia	South Korea	ICRC Busan Office	Kim Min-ji	2023 South Korea Red Cross Annual Work Report	South Korea is a dynamic country in East Asia. The ICRC has offices in Seoul and Busan, focusing on disaster response, community v
Asia	South Korea	ICRC Busan Office	Kim Min-ji	South Korea Typhoon Relief Operation (2023)	South Korea is a dynamic country in East Asia. The ICRC has offices in Seoul and Busan, focusing on disaster response, community v
Asia	Thailand	ICRC Bangkok Office	Somsak Wong	2023 Thailand Red Cross Humanitarian Work Report	Thailand is a Southeast Asian country known for its hospitality. The ICRC operates in Bangkok and Chiang Mai, providing humanitaria
Asia	Thailand	ICRC Bangkok Office	Somsak Wong	Thailand Flood Prevention and Mitigation Project	Thailand is a Southeast Asian country known for its hospitality. The ICRC operates in Bangkok and Chiang Mai, providing humanitaria
+ - ✓ ✕ -- search country relate information -- keypoints: LEFT, RIGHT join SELECT t1.continent_name, t2.country_name, t3.icrc_name, t4.user_name, t5.article_title, t6.'desc' FROM CON... Result Time: 0.003s					
174 records					

The query uses the COUNTRY table as the factual centerpiece. It begins with a RIGHT JOIN to the CONTINENT table, which is a crucial and correct design. This ensures the result includes all country records.

Subsequently, the query switches to a series of LEFT JOIN operations to link to other tables (like IGAC, USER, ARTICLE, etc.). This strategy means: we prioritize obtaining every single country, and then attempt to find related users, articles, descriptions, etc. If this extended information does not exist, the corresponding fields will be NULL, but the country record itself remains in the results. This "core-to-periphery" join chain effectively preserves the integrity of the primary data and prevents information loss due to missing data in the left-side tables.

3.2 Query 2: Search authors who write over one ARTICLE

```

22
23  -- who write over one ARTICLES
24
25  -- Key points: subquery , join
26
27  SELECT
28      t3.user_name,
29      t3.user_country,
30      t4.country_name
31  FROM
32  (
33      SELECT
34          t1.user_name,
35          t1.user_country
36      FROM
37          'USER' t1
38      WHERE
39          t1.user_id IN (
40              SELECT
41                  author_id
42              FROM
43                  'ARTICLE'
44              GROUP BY
45                  author_id
46              HAVING
47                  count( 1 ) > 1
48          )
49      ) t3
50  LEFT JOIN COUNTRY t4 ON t3.user_country = t4.country_id;
51

```

user_name	user_country	country_name	
Tanaka Yuki	2	Japan	
Kim Min-ji	3	South Korea	
Somsak Wong	4	Thailand	
Nguyen Anh	5	Vietnam	
Raj Patel	6	India	
Ahmed Khan	7	Pakistan	
Michael Schmidt	8	Germany	
Pierre Dubois	9	France	
Emma Thompson	10	United Kingdom	
Ivan Petrov	11	Russia	

It decomposes the complex task of "finding users with multiple articles" into two distinct steps. The first step focuses purely on aggregation and filtering within the article table. The second step then fetches the user's details based on the filtered results. This structure makes the intent clear both for human readers and the database optimizer.

Potential Efficiency: Ideally, especially with an index on the author_id field, the inner subquery can be executed very efficiently. It scans the article table once to quickly narrow down the result set through grouping and aggregation before performing any joins. This method prioritizes reducing the dataset size first, which is often more efficient than joining all tables first and then filtering the large intermediate result.

3.3 Query 3: Search the person who write the most amount of ARTICLE

```
local_mysql ms805
61
62 -- The person who writes the most articles with name
63
64 -- rank join agg
65
66 SELECT
67     t2.user_name,
68     count( 1 ) AS ' article num '
69 FROM
70     ARTICLE t1
71     LEFT JOIN 'USER' t2 ON t1.author_id = t2.user_id
72 GROUP BY
73     t1.author_id
74 ORDER BY
75     count( 1 ) DESC
76 LIMIT 100;
77
78
79
80
81
```

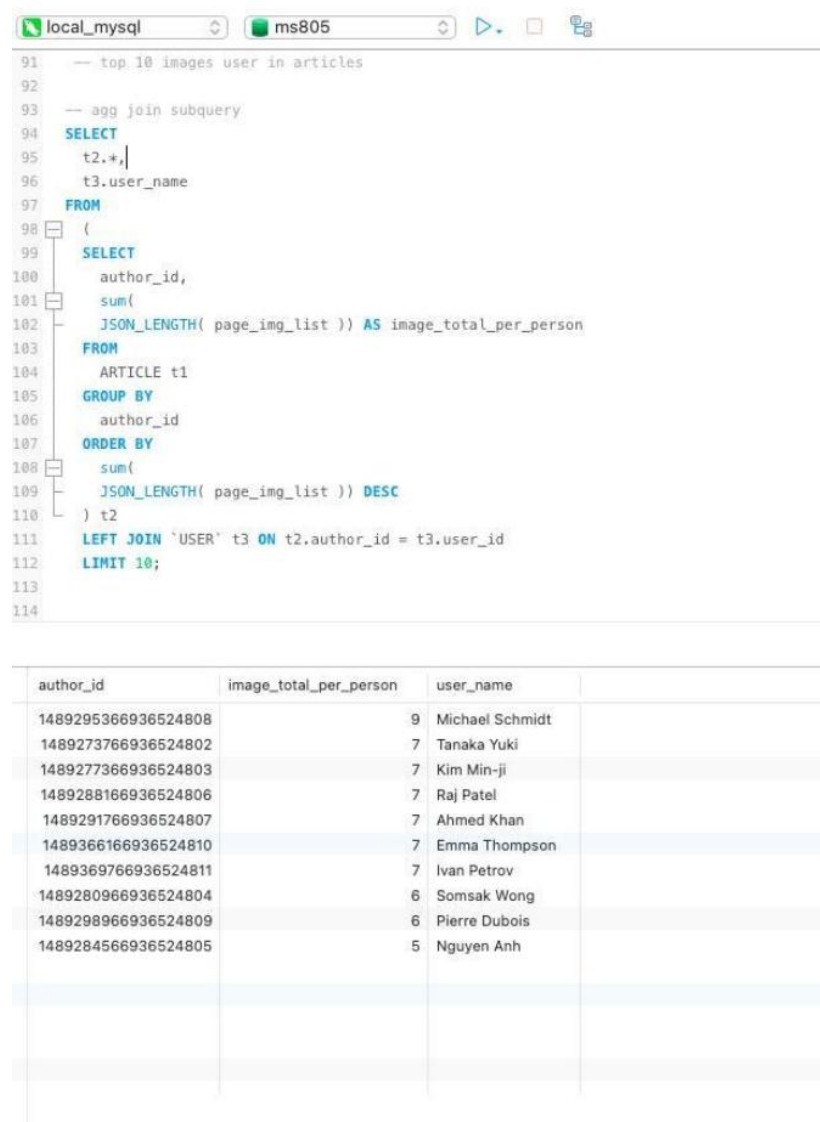
user_name	article num
Michael Schmidt	3
Tanaka Yuki	2
Kim Min-ji	2
Somsak Wong	2
Nguyen Anh	2
Raj Patel	2
Ahmed Khan	2
Pierre Dubois	2
Emma Thompson	2
Ivan Petrov	2
Zhang Wei	1

The entire query logic is straightforward "group and count, then sort by the count in descending order and take the top." It avoids complex subqueries or window functions, making the code easy to write, read, and understand.

High Performance: This query path is very friendly for the database optimizer. It typically requires only one scan of the article table (or even better, using an index on author_id), followed by a single sort of operation after aggregation. Compared to methods that use a subquery to first find the maximum count and then match it, this ORDER BY ... LIMIT pattern is often one of the most performant choices for finding Top N records.

Accurate and Flexible Results: The use of LEFT JOIN ensures that even if an author's information is missing from the USER table (resulting in a NULL user_name), their articles are still counted, guaranteeing the accuracy of the statistics. Furthermore, changing LIMIT 100 to LIMIT 10 easily adapts to the query requests like "get the top 10 authors," demonstrating excellent flexibility.

3.4 Query 4: Search Top 10 Most Used Article Images



The screenshot shows a MySQL query editor with a query window titled 'local_mysql' and a database dropdown set to 'ms805'. The query is as follows:

```
91 -- top 10 images user in articles
92
93 -- agg join subquery
94 SELECT
95     t2.*,
96     t3.user_name
97 FROM
98     (
99     SELECT
100         author_id,
101         sum(
102             JSON_LENGTH( page_img_list )) AS image_total_per_person
103     FROM
104         ARTICLE t1
105     GROUP BY
106         author_id
107     ORDER BY
108         sum(
109             JSON_LENGTH( page_img_list )) DESC
110     ) t2
111 LEFT JOIN `USER` t3 ON t2.author_id = t3.user_id
112 LIMIT 10;
113
114
```

Below the query editor, the results are displayed in a table with the following columns: author_id, image_total_per_person, and user_name. The table shows the top 10 authors based on the number of images used in their articles.

author_id	image_total_per_person	user_name
1489295366936524808	9	Michael Schmidt
1489273766936524802	7	Tanaka Yuki
1489277366936524803	7	Kim Min-ji
1489288166936524806	7	Raj Patel
1489291766936524807	7	Ahmed Khan
1489366166936524810	7	Emma Thompson
1489369766936524811	7	Ivan Petrov
1489280966936524804	6	Somsak Wong
1489298966936524809	6	Pierre Dubois
1489284566936524805	5	Nguyen Anh

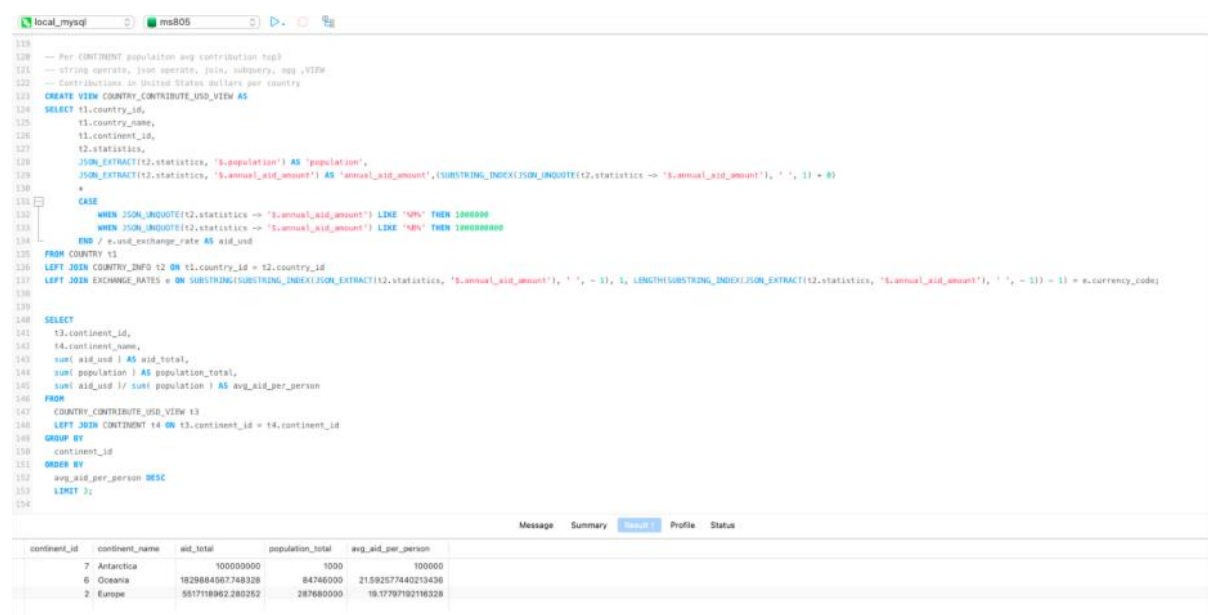
Separation of Concerns: The query encapsulates the computationally "heavy" operations of JSON parsing and aggregation entirely within the inner subquery. This makes the outer query very lightweight and clear, dealing only with a simple join and result of limiting. This structure makes the code's intent very explicit: the inner part focuses on data calculation; the outer part focuses on information enrichment.

Efficient JSON Data Handling: Using the `JSON_LENGTH` function directly to count elements in a JSON array is the best practice for handling this type of data stored in JSON fields. It avoids the need to parse the JSON data into traditional rows and columns, significantly simplifying the query logic and potentially improving performance.

Performance Optimization: Performing the aggregation and sorting inside the subquery means the database only needs to pass a small, pre-sorted intermediate result set (containing only `author_id` and the aggregate value) to the outer query for joining. This is much more efficient than joining all tables first and then performing `JSON_LENGTH` and `SUM` on the large, unaggregated dataset.

Accurate Results: The use of `SUM(JSON_LENGTH(...))` accurately calculates the total image usage per user. The combination of `ORDER BY ... DESC` and `LIMIT 10` perfectly achieves the goal of fetching the "Top 10".

3.5 Query 5: Search Top 3 continents by per capita aid amount



The screenshot shows a MySQL query editor with a SQL query that creates a view and then queries it to find the top 3 continents by per capita aid amount. The query is as follows:

```
119 -- Per CONTINENT population avg contribution top3
120 -- string operators, join operators, join, subquery, agg, VIEW
121 -- Contributions in United States dollars per country
122 CREATE VIEW COUNTRY_CONTRIBUTE_USD_VIEW AS
123 SELECT t1.country_id,
124        t1.country_name,
125        t1.continent_id,
126        t2.statistics,
127        JSON_EXTRACT(t2.statistics, '$.population') AS 'population',
128        JSON_EXTRACT(t2.statistics, '$.annual_aid_amount') AS 'annual_aid_amount',
129        SUBSTRING_INDEX(SUBSTRING_INDEX(SUBSTRING_INDEX(t2.statistics -> '$.annual_aid_amount', ' ', 1) -> 0)
130        *
131        CASE
132            WHEN SUBSTRING_INDEX(SUBSTRING_INDEX(t2.statistics -> '$.annual_aid_amount', ' ', 1) -> 0) LIKE '%%' THEN 1000000
133            WHEN SUBSTRING_INDEX(SUBSTRING_INDEX(t2.statistics -> '$.annual_aid_amount', ' ', 1) -> 0) LIKE '%%' THEN 1000000000
134        END / x.usd_exchange_rate AS aid_usd
135 FROM COUNTRY t1
136 LEFT JOIN COUNTRY_INFO t2 ON t1.country_id = t2.country_id
137 LEFT JOIN EXCHANGE_RATES x ON SUBSTRING_INDEX(SUBSTRING_INDEX(SUBSTRING_INDEX(t2.statistics, '$.annual_aid_amount', ' ', - 1), 1, LENGTH(SUBSTRING_INDEX(SUBSTRING_INDEX(t2.statistics, '$.annual_aid_amount', ' ', - 1)) - 1)) = x.currency_codes);
138
139
140 SELECT
141     t3.continent_id,
142     t4.continent_name,
143     sum_aid_usd / AS aid_total,
144     sum_population / AS population_total,
145     sum_aid_usd / sum_population / AS avg_aid_per_person
146 FROM
147     COUNTRY_CONTRIBUTE_USD_VIEW t3
148 LEFT JOIN COUNTRY t4 ON t3.continent_id = t4.continent_id
149 GROUP BY
150     continent_id
151 ORDER BY
152     avg_aid_per_person DESC
153 LIMIT 3;
154
```

The results of the query are shown in a table with the following columns: `continent_id`, `continent_name`, `aid_total`, `population_total`, and `avg_aid_per_person`.

continent_id	continent_name	aid_total	population_total	avg_aid_per_person
7	Antarctica	100000000	1000	100000
8	Oceania	1928684187748209	94760000	2152577440213498
2	Europe	9517118962280262	287680000	181797192146328

This creating a view encapsulates the complex logic of data cleansing, parsing, and currency conversion into a single, named module. This makes the final query clean and easy to understand, focusing only on aggregation and sorting. The view itself becomes a reusable asset for other queries needing standardized country-level aid data.

Effective Handling of Complex Data: The query adeptly handles data stored in JSON with combined number-unit strings. The strategic combination of `JSON_EXTRACT`, `SUBSTRING_INDEX`, and `CASE WHEN` is a powerful and practical data wrangling technique to derive meaningful numbers from unstructured text.

Accurate Currency Conversion: By joining the exchange rate table, it ensures accurate conversion, allowing for a fair comparison and summation of aid amounts across different currencies. This guarantees accuracy and comparability of the result.

Correct Aggregation Logic: The calculation for the per-capita value uses SUM (total) / SUM (population) instead of AVG (per_capita). This is the statistically correct method as it accounts for the varying population sizes of different countries, preventing small countries from skewing the overall average.

4.Summary

4.1 Core Functionality Support

This database design comprehensively addresses the core business requirements of the ICRC "Where We Work" page. In terms of content management, the system supports the management of articles and user data through dedicated table structures. It not only records the basic information of articles but also maintains the relationships between articles and their authors, as well as articles and countries, ensuring orderly content organization.

In geographic information management, the database clearly constructs a hierarchical structure between continents and countries. This design enables the website to implement basic navigation features for browsing countries by region and lays out the foundation for subsequent regional data analysis.

For institutional data, the system specifically stores information about ICRC offices in various countries, including key data such as office names and geographic locations. This directly supports the implementation of the website's map feature, allowing users to intuitively understand ICRC's global distribution, then, through carefully designed foreign key constraints, the system effectively maintains the relational integrity between different data entities, ensuring data consistency and accuracy while preventing the existence of orphaned data.

4.2 Technical Features

In terms of technical implementation, this system adopts standard SQL syntax for development, ensuring high compatibility with mainstream database management systems. It can run smoothly on various common database platforms, whether MySQL, PostgreSQL, or others.

The database's relationship design reflects a clear architectural approach. The relationships between tables are carefully planned, meeting current business needs while maintaining sufficient flexibility to adapt to future changes.

The business query scenarios provided by the system are highly practical, covering various use cases from simple content retrieval to complex data analysis. These queries can directly support the implementation of the website's core functionalities.

Furthermore, the system's architectural design considers long-term maintenance needs. By adopting a modular design philosophy, subsequent functional expansions and adjustments to table structures can be achieved at relatively low costs, demonstrating good maintainability and extensibility.